

FormalRace-Checker: Um Verificador Formal de Corridas de Dados para Código Concorrente em C Baseado em SAT Solving

Sarah C. A. Pereira¹

¹ Departamento de Ciência da Computação
Universidade Federal de Roraima (UFRR)
Boa Vista – RR – Brasil

sarahscarolinec@gmail.com

Abstract. *Programs that use POSIX Threads (pthreads) are common in operating systems, embedded software, and real-time applications. In these systems, incorrect use of synchronization primitives can lead to data races: situations where two threads access the same memory location without proper protection, and at least one of the accesses is a write. Such faults produce unpredictable behavior and often escape conventional testing.*

This paper presents FormalRace-Checker, a static analysis tool that extracts information from C programs with pthreads, translates concurrency conditions into logical formulas in Conjunctive Normal Form, and submits them to the MiniSAT solver. When the solver returns satisfiable, the tool generates a report identifying the threads, variables, and source code lines involved.

The tool was validated using three test scenarios: no protection, correct mutual exclusion, and nested locks with different acquisition orders. In all cases, the obtained results matched the expected outcomes. The source code is publicly available and implemented in C++11.

Resumo. *Programas que utilizam POSIX Threads (pthreads) são comuns em sistemas operacionais, software embarcado e aplicações de tempo real. Nesses sistemas, o uso incorreto de primitivas de sincronização pode levar a corridas de dados: situações em que duas threads acessam a mesma posição de memória sem proteção adequada, sendo pelo menos um desses acessos uma escrita. Esse tipo de falha produz comportamento imprevisível e costuma escapar de testes convencionais.*

Este artigo apresenta o FormalRace-Checker, uma ferramenta de análise estática que extrai informações de programas C com pthreads, traduz as condições de concorrência em fórmulas lógicas na Forma Normal Conjuntiva e as submete ao solver MiniSAT. Quando o solver indica satisfatibilidade, a ferramenta gera um relatório apontando as threads, variáveis e linhas de código envolvidas.

A validação da ferramenta foi feita com três cenários de teste: ausência total de proteção, exclusão mútua bem aplicada e uso de múltiplos locks com ordens de aquisição distintas. Em todos os casos, o resultado obtido foi o esperado. O código fonte está disponível publicamente e foi implementado em C++11.

1. Introdução

A crescente disponibilidade de processadores multi-core tornou a programação concorrente uma prática cotidiana no desenvolvimento de sistemas de infraestrutura. Sistemas de controle automotivo, *kernels* de sistemas operacionais embarcados e plataformas de processamento de dados em tempo real dependem do modelo de múltiplas linhas de execução (*multithreading*) sobre memória compartilhada para atingir requisitos de desempenho e responsividade [Silberschatz et al. 2018].

O modelo de programação baseado em *POSIX Threads* (pthreads) fornece primitivas de sincronização como *mutexes*, semáforos e variáveis de condição. Contudo, o gerenciamento manual dessas primitivas é altamente propenso a falhas humanas. O erro mais crítico desse domínio é o *data race* (corrida de dados): ocorre quando duas ou mais *threads* acessam concorrentemente a mesma posição de memória sem sincronização comum, sendo que pelo menos um dos acessos é uma operação de escrita [Clarke et al. 2018].

Corridas de dados produzem *Heisenbugs* — falhas cujo comportamento se altera ou desaparece sob inspeção —, tornando-as resistentes a métodos convencionais de teste de caixa-preta [Flanagan and Freund 2009]. Ferramentas de análise dinâmica como *Helgrind* e *ThreadSanitizer* detectam *races* apenas nas interleavings que ocorrem durante a execução monitorada, deixando o espaço remanescente de interleavings não analisado.

O **FormalRace-Checker**, proposto neste trabalho, é uma ferramenta de verificação estática que transforma o problema da detecção de corridas de dados em um problema de satisfatibilidade booleana (SAT). A abordagem garante que *todos* os interleavings possíveis sejam considerados matematicamente, independentemente de qual ocorre em uma execução específica.

O repositório do projeto está disponível em: https://github.com/sarahw3b/FP_DCC403_Tema_8_UFRR

2. Fundamentação Teórica

2.1. Semântica de Entrelaçamento

Em um sistema com n threads $T_1, T_2, \dots, T_n$, a execução concorrente é modelada como o conjunto de todas as interleavings possíveis das instruções individuais. Define-se o conjunto de eventos abstratos de uma thread como:

$$E = \{lock(m), unlock(m), read(x), write(x)\} \quad (1)$$

onde m denota um identificador de *mutex* e x uma variável global compartilhada [Baier and Katoen 2008]. Cada execução concorrente corresponde a uma sequência linear de eventos de E gerada pelo escalonador do sistema operacional.

2.2. Definição Formal de Data Race

Dados dois acessos A_i (pela thread T_i) e A_j (pela thread T_j) à mesma variável x , com $i \neq j$, ocorre uma corrida de dados se e somente se: (i) pelo menos um dos acessos é uma operação de escrita; e (ii) os dois acessos não estão protegidos por um *mutex* comum.

Para o par composto por uma escrita W_1 em T_1 e uma leitura R_2 em T_2 sobre a variável x , define-se:

$$Race(W_1, R_2) \iff W_1 \wedge R_2 \wedge \neg P(W_1, R_2) \quad (2)$$

onde $P(W_1, R_2)$ é verdadeiro se e somente se ambos os acessos estão protegidos pelo mesmo identificador de exclusão mútua [Kroening and Strichman 2016].

2.3. Satisfatibilidade Booleana (SAT)

Um SAT Solver recebe uma fórmula proposicional em Forma Normal Conjuntiva (CNF) e determina se existe alguma atribuição de valores-verdade às variáveis que satisfaça todas as cláusulas simultaneamente. O formato padrão de entrada é o DIMACS CNF:

- Cabeçalho: `p cnf <num_vars> <num_clausulas>`
- Cada cláusula é uma linha de literais inteiros terminada por 0
- Inteiros positivos representam variáveis afirmadas; negativos, negadas

Associando à Equação 2 os índices proposicionais $1 \equiv W_1$, $2 \equiv R_2$ e $3 \equiv P(W_1, R_2)$, a codificação DIMACS correspondente é:

```
p cnf 3 3
1 0
2 0
-3 0
```

Listing 1. Codificação CNF para detecção de race condition

Se o solver retorna SAT, existe uma valoração onde $W_1 = \top$, $R_2 = \top$ e $P = \perp$, confirmando a corrida de dados. Se retorna UNSAT, o código está provado seguro para o par analisado [Eén and Sörensson 2003].

3. Arquitetura do FormalRace-Checker

A ferramenta é organizada em quatro módulos independentes que compõem o pipeline de verificação:

[Código C] $\rightarrow$ Parser $\rightarrow$ Gerador CNF $\rightarrow$ SAT Solver $\rightarrow$ [Relatório]

3.1. Módulo 1 — Parser Estático

O parser realiza três passagens sobre o arquivo `.c`:

Passagem 1 — Variáveis globais: varre o arquivo contando o balanço de chaves `{}`. Declarações de variáveis encontradas com balanço igual a zero (fora de qualquer função) são registradas como candidatas a corrida de dados. Declarações do tipo `pthread_mutex_t` são explicitamente ignoradas por não constituírem dados compartilhados.

Passagem 2 — Funções de thread: busca chamadas a `pthread_create()` e extrai o terceiro argumento (nome da função de thread) via expressão regular.

Passagem 3 — Acessos e escopos de mutex: para cada função de thread, varre suas linhas mantendo uma pilha de *mutexes* ativos: empilha no `pthread_mutex_lock()` e desempilha no `pthread_mutex_unlock()`. Detecta escritas (operadores `=`, `+=`, `-=`, `++`, `--`) e leituras de variáveis globais, registrando o *mutex* do topo da pilha para cada acesso.

O uso de uma pilha em vez de uma única variável de estado permite rastrear corretamente *locks* aninhados — cada acesso é associado ao *mutex* mais interno que o protege.

3.2. Módulo 2 — Gerador de Fórmulas CNF

Para cada variável global, o módulo forma todos os pares (W_i, R_j) com $i \neq j$. Para cada par, verifica se $W_i.mutex = R_j.mutex \neq \cdot$. Pares que não satisfazem essa condição são inseguros e geram três cláusulas DIMACS.

Quando não há pares inseguros, o módulo grava `p cnf 0 0` (fórmula vazia). O Módulo 3 interpreta essa condição como UNSAT sem invocar o solver, evitando o falso positivo que o MiniSAT produziria ao receber uma fórmula trivialmente satisfatível sem cláusulas.

3.3. Módulo 3 — Solver Connector

O módulo lê o cabeçalho do arquivo CNF para verificar o número de cláusulas. Se for zero, retorna `false` imediatamente. Caso contrário, invoca o MiniSAT via `system()` com redirecionamento de saída e lê o arquivo de resultado:

```
1 string cmd = "minisat " + cnfFile + " " + resultFile
2             + " > /dev/null 2>&1";
3 system(cmd.c_str());
```

Listing 2. Invocação do MiniSAT

A comparação do resultado usa igualdade exata (`line == "SAT"`) em vez de busca por substring, evitando que "UNSAT" case como "SAT" por conter a sequência de caracteres.

3.4. Módulo 4 — Sintetizador de Relatórios

Quando o solver retorna SAT, o relatório exibe: (i) todos os acessos extraídos pelo parser; (ii) os pares conflitantes com `thread`, linha e `mutex` de cada acesso; (iii) confirmação da ausência de `mutex` comum; e (iv) sugestão de correção com exemplo de código. Quando retorna UNSAT, confirma formalmente que o código está seguro para execução concorrente dentro do escopo analisado.

4. Implementação

A ferramenta foi implementada em C++11, compilada com `g++ -std=c++11 -Wall`, e validada no ambiente Linux Fedora com interface gráfica KDE Plasma. O solver utilizado é o MiniSAT 2.2 [Eén and Sörensson 2003], instalado via repositório oficial (`dnf install minisat`). O projeto é organizado nos seguintes arquivos:

- `src/parser.h / parser.cpp` — Módulo 1
- `src/cnf_gen.h / cnf_gen.cpp` — Módulo 2
- `src/solver.h / solver.cpp` — Módulo 3
- `src/reporter.h / reporter.cpp` — Módulo 4
- `main.cpp` — ponto de entrada e orquestração
- `tests/` — cenários de teste obrigatórios

A execução é feita via linha de comando:

```
./formalrace-checker <arquivo.c>
```

5. Estudo de Casos e Resultados

5.1. Cenário A — Acesso sem Proteção

O código `tests/cenario_a.c` contém duas *threads* acessando a variável global `contador`: a função `writer` realiza escrita na linha 7 sem *mutex*, e a função `reader` realiza leitura na linha 12 também sem *mutex*. O parser identificou o par desprotegido, o Módulo 2 gerou um arquivo CNF com 3 variáveis e 3 cláusulas, e o MiniSAT retornou SAT. Saída da ferramenta:

```
Par conflitante: writer escreve, reader le em contador
                (sem mutex comum)
CNF gerado em output.cnf (3 variaveis, 3 clausulas)
RESULTADO: DATA RACE DETECTADO!

- RELATORIO DE ANALISE: -
DATA RACE DETECTADO!
ACESSOS EXTRAIDOS:
  Thread: writer, Variavel: contador, Tipo: write,
          Linha: 7, Mutex: nenhum
  Thread: reader, Variavel: contador, Tipo: read,
          Linha: 12, Mutex: nenhum
PARES CONFLITANTES:
  Escrita: writer (linha 7) em 'contador' | Mutex: nenhum
  Leitura: reader (linha 12) em 'contador' | Mutex: nenhum
-> Nenhum mutex comum protege ambos os acessos.
```

Listing 3. Saída — Cenário A

5.2. Cenário B — Sincronização Homologada

O código `tests/cenario_b.c` protege todos os acessos à variável `contador` com o mesmo *mutex*. Para cada par analisado, a condição $A_i.mutex = A_j.mutex$ foi satisfeita, resultando em zero cláusulas no CNF. O Módulo 3 identificou a fórmula vazia e retornou false sem invocar o solver. Saída:

```
Nenhum par conflitante encontrado. Sem data races.
RESULTADO: Nenhum data race encontrado.

- RELATORIO DE ANALISE: -
Nenhum data race encontrado.
O codigo esta seguro para execucao concorrente.
```

Listing 4. Saída — Cenário B

5.3. Cenário C — Locks Aninhados Assimétricos

O código `tests/cenario_c.c` utiliza dois *mutexes* (`mutex1` e `mutex2`) e duas variáveis globais (`saldo` e `estoque`). A `thread1` adquire `mutex1` depois `mutex2`; a `thread2` adquire `mutex2` depois `mutex1`. Embora essa ordem assimétrica configure risco de *deadlock*, o parser rastreou os escopos por meio da pilha de *locks* e verificou que cada variável global é sempre acessada dentro do escopo do mesmo *mutex* nas duas *threads*: `saldo` sob `mutex1` e `estoque` sob `mutex2`. Resultado: UNSAT — código seguro.

```
Nenhum par conflitante encontrado. Sem data races.
RESULTADO: Nenhum data race encontrado.

- RELATORIO DE ANALISE: -
Nenhum data race encontrado.
O codigo esta seguro para execucao concorrente.
```

Listing 5. Saída — Cenário C

5.4. Resumo dos Resultados

A Tabela 1 consolida os resultados da suíte de testes obrigatória:

Tabela 1. Resultados da suíte de testes obrigatória			
Cenário	Esperado	Obtido	Acerto
A — Sem proteção mutex	SAT	SAT	✓
B — Mutex correto	UNSAT	UNSAT	✓
C — Locks aninhados	UNSAT	UNSAT	✓

6. Discussão e Limitações

A abordagem adotada representa uma simplificação intencional em relação a verificadores formais industriais. As limitações são explicitadas para delimitar o escopo da ferramenta:

Acesso via ponteiros: o parser textual não rastreia ponteiros para variáveis globais. Uma escrita do tipo `*p = 42`, onde `p = &global_x`, não é detectada, constituindo um falso negativo.

Análise interprocedural: se a aquisição de *mutex* ocorre dentro de uma função auxiliar chamada pela *thread* (e não diretamente no corpo da função de *thread*), o parser não associa o escopo de *lock* ao acesso, podendo gerar falsos positivos.

Explosão combinatória: o número de pares cresce quadraticamente com o número de acessos por variável. Para bases de código muito grandes, o tempo de resolução pode se tornar proibitivo. Nos cenários avaliados (até 2 *threads* e 2 variáveis globais), o tempo total de execução foi inferior a 100ms.

Essas limitações definem o escopo da ferramenta como adequado para auditorias em código de pequeno a médio porte com padrões explícitos de sincronização.

7. Aplicação Prática

Uma aplicação prática direta do FormalRace-Checker é a auditoria de código em sistemas embarcados de tempo real, como controladores de acesso a sensores industriais. Nesses sistemas, múltiplas tarefas concorrentes atualizam variáveis de estado compartilhadas (leituras de temperatura, pressão, status de atuadores) e qualquer corrida de dados pode gerar leituras incorretas ou comandos inconsistentes.

A integração da ferramenta em um *pipeline* de CI/CD (*Continuous Integration*) permitiria que todo *commit* que modifique arquivos de lógica concorrente seja automaticamente auditado antes de ser integrado à base de código. A saída estruturada do relatório facilita a geração de alertas automáticos para a equipe de desenvolvimento.

8. Trabalhos Relacionados

RacerX [Engler and Ashcraft 2003] realiza análise estática de *races* e *deadlocks* em código de *kernel* usando inferência de fluxo de controle interprocedural, com precisão superior à abordagem textual adotada neste trabalho. *FastTrack* [Flanagan and Freund 2009] é uma abordagem dinâmica eficiente baseada em relógios de vetor que detecta *races* apenas nas interleavings observadas em tempo de execução. O verificador CBMC [Kroening and Strichman 2016] realiza *bounded model checking* de programas C com semântica completa de ponteiros e análise interprocedural.

Em contraste com essas ferramentas, o FormalRace-Checker prioriza a simplicidade de instalação (dependência única: MiniSAT), a transparência do processo de verificação (arquivo CNF inspecionável) e o valor pedagógico da tradução explícita de propriedades de concorrência em fórmulas booleanas.

9. Conclusão

Este trabalho apresentou o FormalRace-Checker, uma ferramenta de verificação formal estática para detecção de corridas de dados em programas C com pthreads. A ferramenta implementa um pipeline de quatro módulos que comprova ou descarta matematicamente a ocorrência de *data races* sem executar o programa analisado.

Os três cenários de teste obrigatórios foram resolvidos com 100% de precisão. O Cenário A confirmou a detecção de um acesso desprotegido (SAT). Os Cenários B e C confirmaram a aprovação de código corretamente sincronizado (UNSAT), incluindo o caso de *locks* aninhados assimétricos, que é tratado corretamente pela pilha de *mutexes* do parser.

Como trabalho futuro, propõe-se a extensão do parser para análise interprocedural via grafo de chamadas e o rastreamento de acessos via ponteiros, aproximando a ferramenta do conjunto de funcionalidades de verificadores industriais como CBMC e RacerX.

Declaração de Uso de Inteligência Artificial

A autora declara que, durante o desenvolvimento deste trabalho, foram utilizadas as seguintes ferramentas de inteligência artificial generativa:

- **Claude** (Anthropic) — apoio na estruturação do pipeline de verificação, revisão de código C++ e aprimoramento da redação técnica do artigo;
- **DeepSeek** (DeepSeek AI) — consultas complementares sobre o formato DIMACS e integração com o MiniSAT;

Todo o conteúdo conceitual, o código-fonte do projeto, os cenários de teste e a redação final foram supervisionados, validados e são de responsabilidade exclusiva da autora. O uso das ferramentas acima restringiu-se a apoio técnico e revisão, sem substituição do raciocínio autoral.

Referências

- [Baier and Katoen 2008] Baier, C. and Katoen, J.-P. (2008). *Principles of Model Checking*. MIT Press.

- [Clarke et al. 2018] Clarke, E. M., Grumberg, O., Kroening, D., Peled, D., and Veith, H. (2018). *Model Checking*. MIT Press, 2nd edition.
- [Eén and Sörensson 2003] Eén, N. and Sörensson, N. (2003). An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT)*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer.
- [Engler and Ashcraft 2003] Engler, D. and Ashcraft, K. (2003). RacerX: Effective, static detection of race conditions and deadlocks. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pages 237–252. ACM.
- [Flanagan and Freund 2009] Flanagan, C. and Freund, S. N. (2009). FastTrack: Efficient and precise dynamic race detection. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 121–133. ACM.
- [Kroening and Strichman 2016] Kroening, D. and Strichman, O. (2016). *Decision Procedures: An Algorithmic Point of View*. Springer, 2nd edition.
- [Silberschatz et al. 2018] Silberschatz, A., Galvin, P. B., and Gagne, G. (2018). *Operating System Concepts*. Wiley, 10th edition.